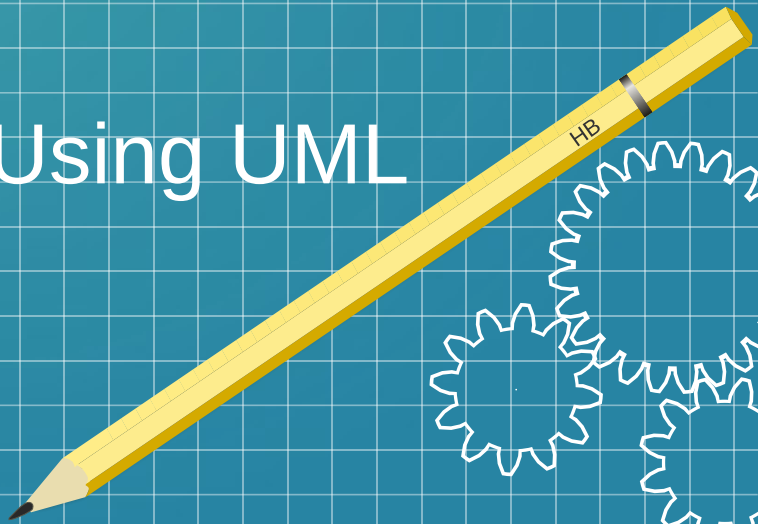
CSC-300-Software Engineering

UNIT-III: Object Modelling Using UML



Model

- **A model is a simplified version of a real system.**
- A model is **constructed by focusing only on a few aspects of the problem and ignoring the rest**.
- **The model of a problem is called an *analysis model***. On the other hand, **the model of the solution (code) is called the *design model*.**

Unified Modelling Language (UML)

- **A model can be documented using a modelling language such as Unified Modelling Language(UML).**
- UML has been **accepted by ISO as a standard for modelling object-oriented systems**.
- UML can be used to **document object-oriented analysis and design results that have been obtained using any methodology**.

UML Diagrams

- **UML 1.0 can be used to construct nine different types of diagrams to capture five different views of a system**.
- The different UML diagrams **provide different perspectives of a software system to be developed** and **facilitate a comprehensive understanding of the system**.
- Each perspective focuses on some specific aspect and ignores the rest.

UML Diagrams – Views

- UML diagrams can capture the following views (models) of a system:
  - **User's view**
  - **Structural view**
  - **Behaviourial view**
  - **Implementation view**
  - **Environmental view**

UML Diagrams – Views Diagram

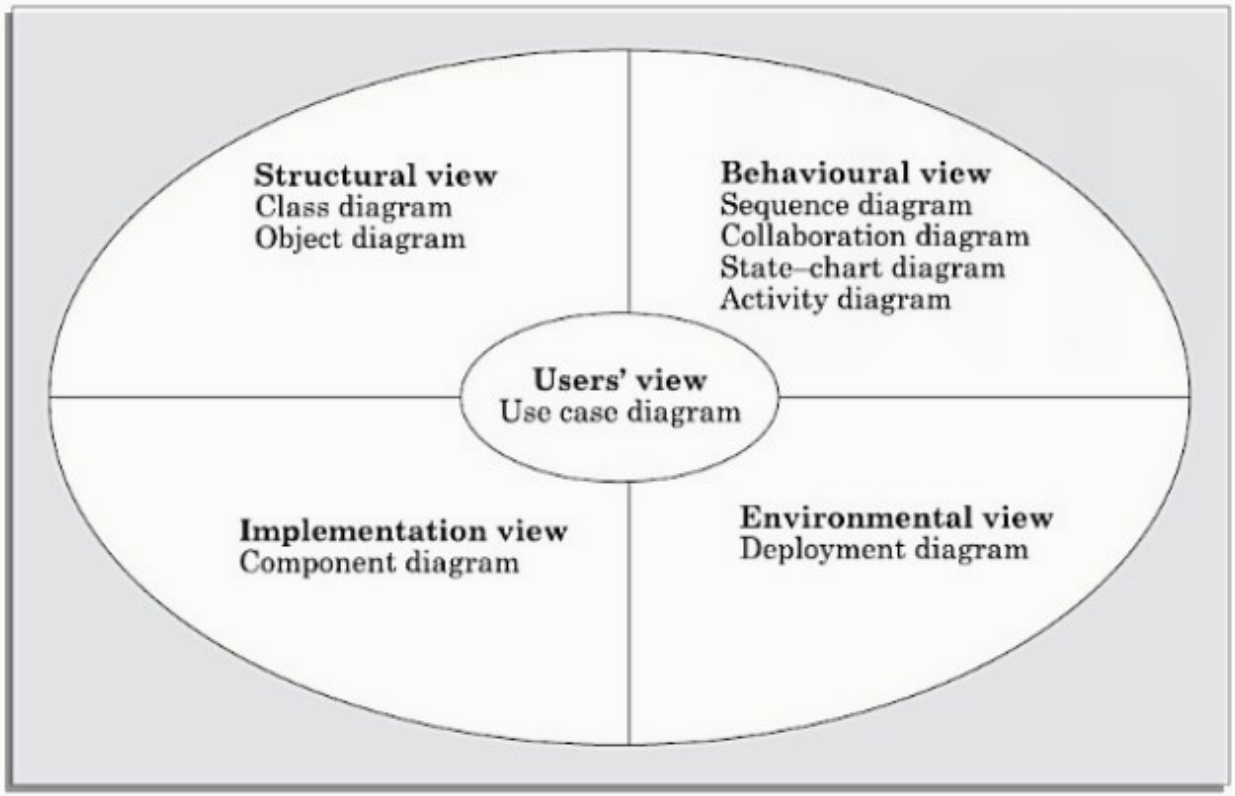


Figure 7.14: Different types of diagrams and views supported in UML.

UML Diagrams – Views Diagram

- Users' view is shown as the central view because **based on the users' view, all other views are developed.**
- All the **other views need to conform to the user's view**.
- Most of the object oriented analysis and design methodologies require us to **iterate among the different views several times to arrive at the final design**.

UML Diagrams – User's View

- This view **defines the functionalities made available by the system to its users**.
- **The users' view is a black-box view of the system** where the internal structure, the dynamic behaviour of different system components, the implementation etc. are not captured.
- **It is a functional model compared to all other views that are essentially object models**.

UML Diagrams – Structural View

- The structural view **defines the structure of the problem (or the solution) in terms of the kinds of objects (classes) important to the understanding of the working of a system and to its implementation**.
- It also **captures the relationships among the classes (objects)**.
- **The structural model is also called the *static model***.

UML Diagrams – Behaviourial View

- The behaviourial view **captures how objects interact with each other in time to realise the system behaviour**.
- **The system behaviour captures the time-dependent (dynamic) behaviour of the system**.
- **It therefore constitutes the dynamic model of the system**.

UML Diagrams – Implementation View

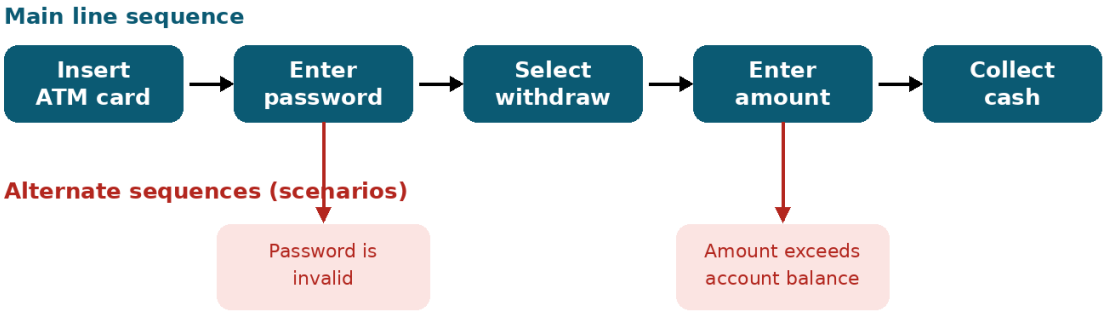
- This view **captures the important components of the system and their interdependencies**.
- For example, the implementation view might show the GUI part, the middleware, and the database part as the different parts and also would capture their interdependencies

UML Diagrams – Environmental View

- This view **models how the different components are implemented on different pieces of hardware**.

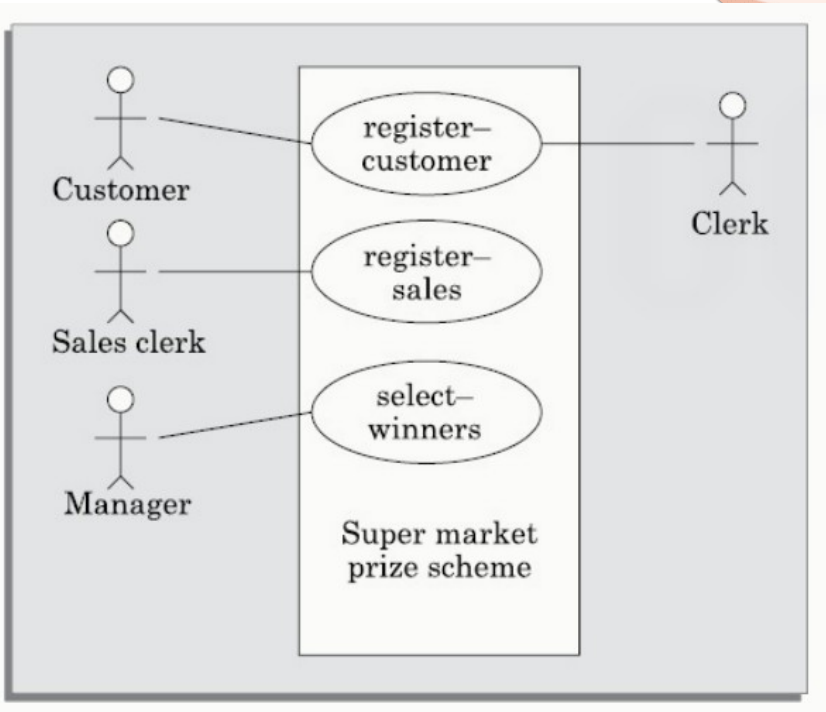
Use Case Model

- The use case model is **a set of use cases** showing the **different ways a system can be used**.
- A use case defines **coherent behaviour without revealing internal structure**.
- Each use case has **one *main line sequence*** and **several alternate sequences**; each is a **scenario**.



Use Case Diagram

- **Use case** = ellipse; **system boundary** = rectangle.
- **Actor** = stick person: a **role** played by a user or an <<external system>>.
- The line between them is the **communication relationship**.



Class Diagrams

- A class diagram shows the **static structure** of a system.
- Main parts: **classes** and their **relationships**.
- Class = **rectangle with compartments**; the **name compartment** is mandatory.
- Class names: **singular nouns**, e.g. ***LibraryMember***.

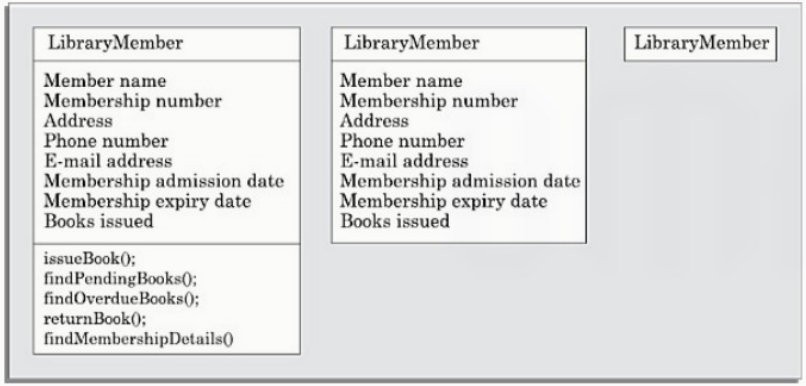
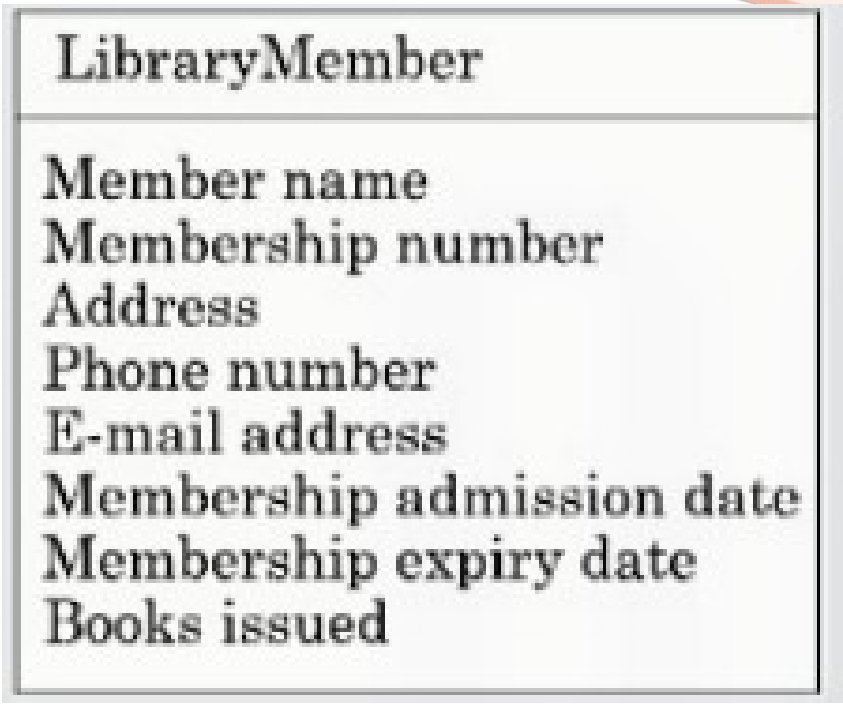


Figure 7.23: Different representations of the LibraryMember class.

Class Diagrams – Attributes

- An attribute is **a named property of a class**.
- Names begin with a **lower-case letter**.
- Optional **type**, **initial value**, **multiplicity**.
- Example: ***sensorStatus[1]:Int=0***



Class Diagrams – Operations

- Operation names begin with a **lower-case letter**.
- **Abstract operations** are written in **italics**.
- Parameter kinds: **in**, **out**, **inout** (default **in**).
- Example: ***issueBook(in bookName):Boolean***



Class Diagrams – Association

- A **straight line** joins the associated classes.
- An **arrowhead** shows the **reading direction**.
- **Multiplicity** at each end: number, range **1..5**, or * (zero or more).



Figure 7.24: Association between two classes.

Class Diagrams – Aggregation

- A **whole-part** special case of association.
- Shown by an **empty diamond** at the aggregate end.
- **Cannot be reflexive or symmetric**, but can be **transitive**.



Figure 7.25: Representation of aggregation.

Class Diagrams – Composition

- A **stricter aggregation**: parts are **existence-dependent** on the whole.
- **Parts are created and destroyed with the whole.**
- Shown by a **filled diamond** at the composite end.



Figure 7.26: Representation of composition.

Class Diagrams – Inheritance

- An **empty arrow** points from **subclass to superclass**.
- A **discriminator** labels the aspect abstracted.
- Subclasses sharing a discriminator form a ***partition***.

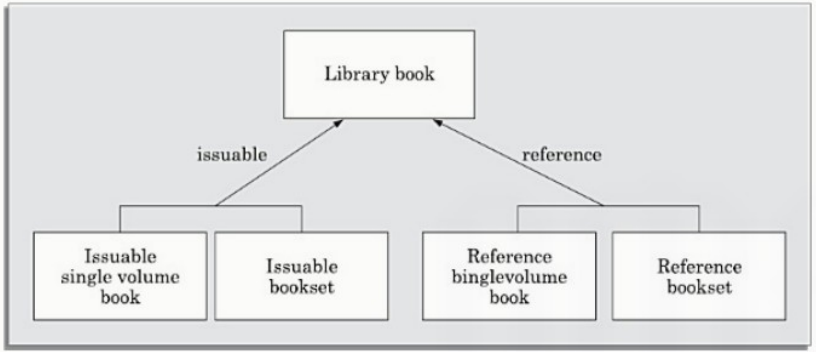


Figure 7.27: Representation of the inheritance relationship.

Dependency & Constraints

- **Dependency**: dotted arrow from the **dependent** to the **independent** class.
- **Constraints** limit attribute values, set **pre-/post-conditions** or ordering.
- Example: **{sorted}** on Book (ISBN order).

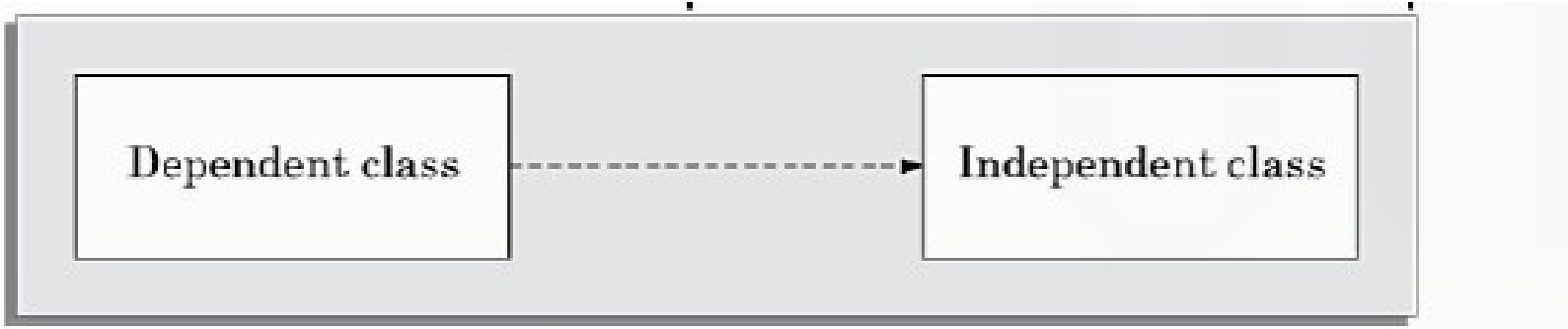


Figure 7.28: Representation of dependence between classes.

Object Diagrams

- Show a **snapshot of objects at a point in time**.
- Also called an ***instance diagram***.
- Objects are drawn as **rounded rectangles**.

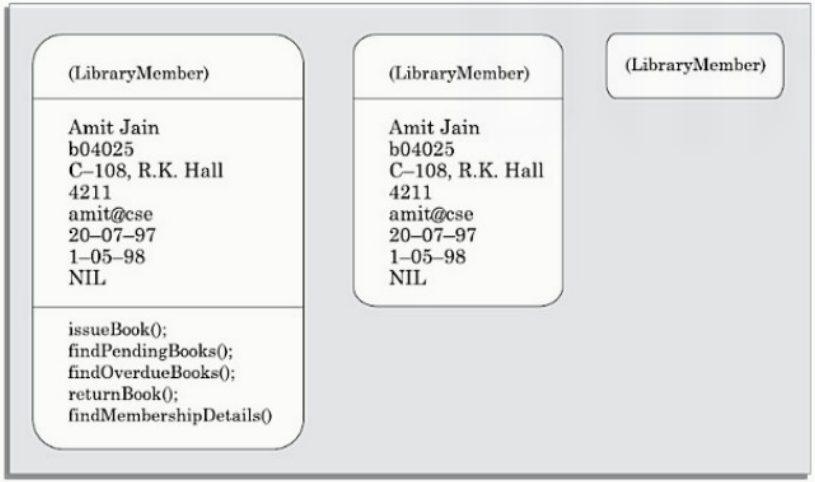


Figure 7.29: Different representations of a LibraryMember object.

Interaction Diagram

- **Interaction diagrams are models that describe how groups of objects interact among themselves** through message passing to realise some behaviour or accomplish some task.

- Typically, **each interaction diagram realises the behaviour of a single use case**.

- **For complex use cases, more than one interaction diagrams may be necessary** to capture the behaviour.

- There are two kinds of interaction diagrams:
  - **Sequence diagrams** and
  - **Collaboration diagrams**
- These two actually **portray different perspectives of some behaviour of a system** and **different types of inferences can be drawn from them**.

Interaction Diagram – *Sequence Diagram*

- A sequence diagram **shows interaction among objects as a two dimensional chart**.

- **The objects participating in the interaction are shown at the top of the chart** as boxes attached to a vertical dashed line.

- An object appearing at the top of the sequence diagram signifies that the object existed even before the time the use case execution was initiated.

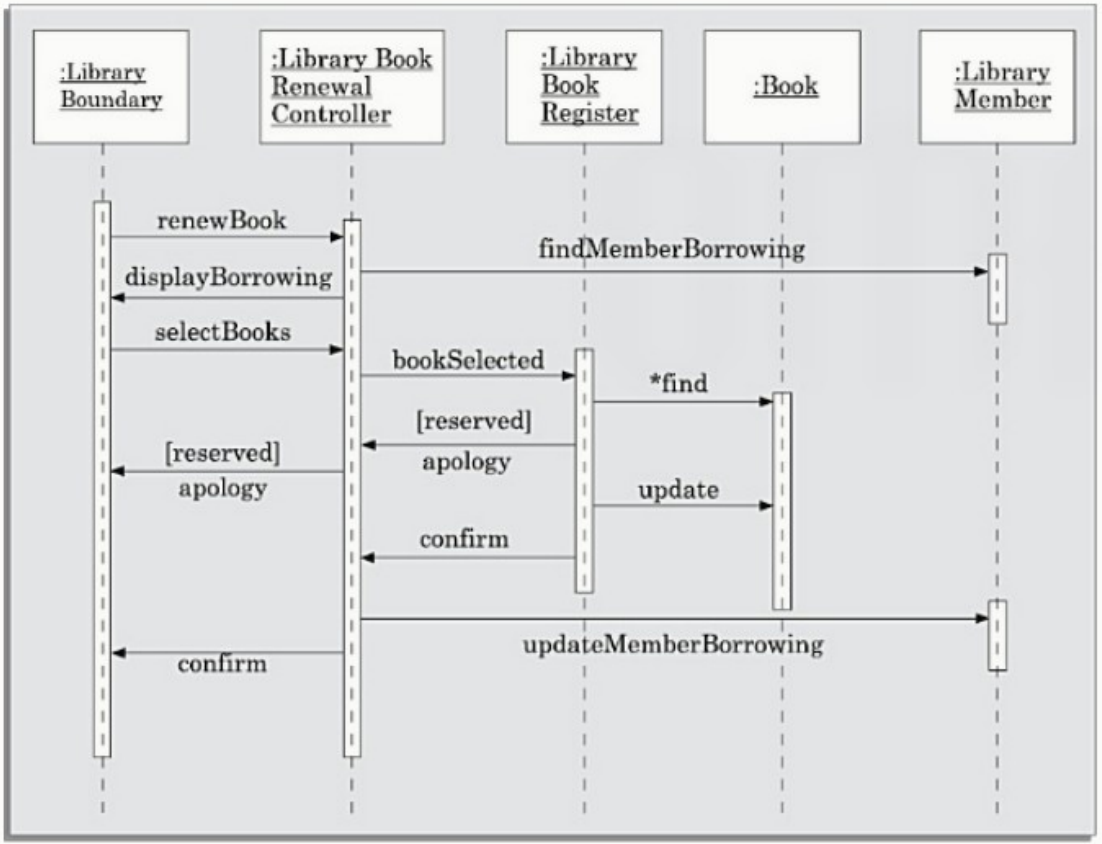


Figure 7.30: Sequence diagram for the renew book use case

- The vertical dashed line is called the ***object's lifeline***. Absence of lifeline after some point indicates that the object ceases to exist after that point in time.

- A rectangle called the ***activation symbol*** is drawn on the lifeline to indicate the points of time at which the object is active.

- **Each message is indicated as an arrow between the lifelines of two objects.** The messages are shown in chronological order from the top to the bottom.

- Some control information can also be included.
  - `[condition]`
`message` — means message will be sent only when condition is true
  - `[for every object]`
`message` — means message will be sent for each object in some collection

Collaboration Diagram

- **A collaboration diagram shows both structural and behavioural aspects explicitly.** Unlike a sequence diagram which shows only the behavioural aspects.

- The **structural aspect of a collaboration diagram consists of objects and links among them indicating association**.

- Each object is also called a ***collaborator***.

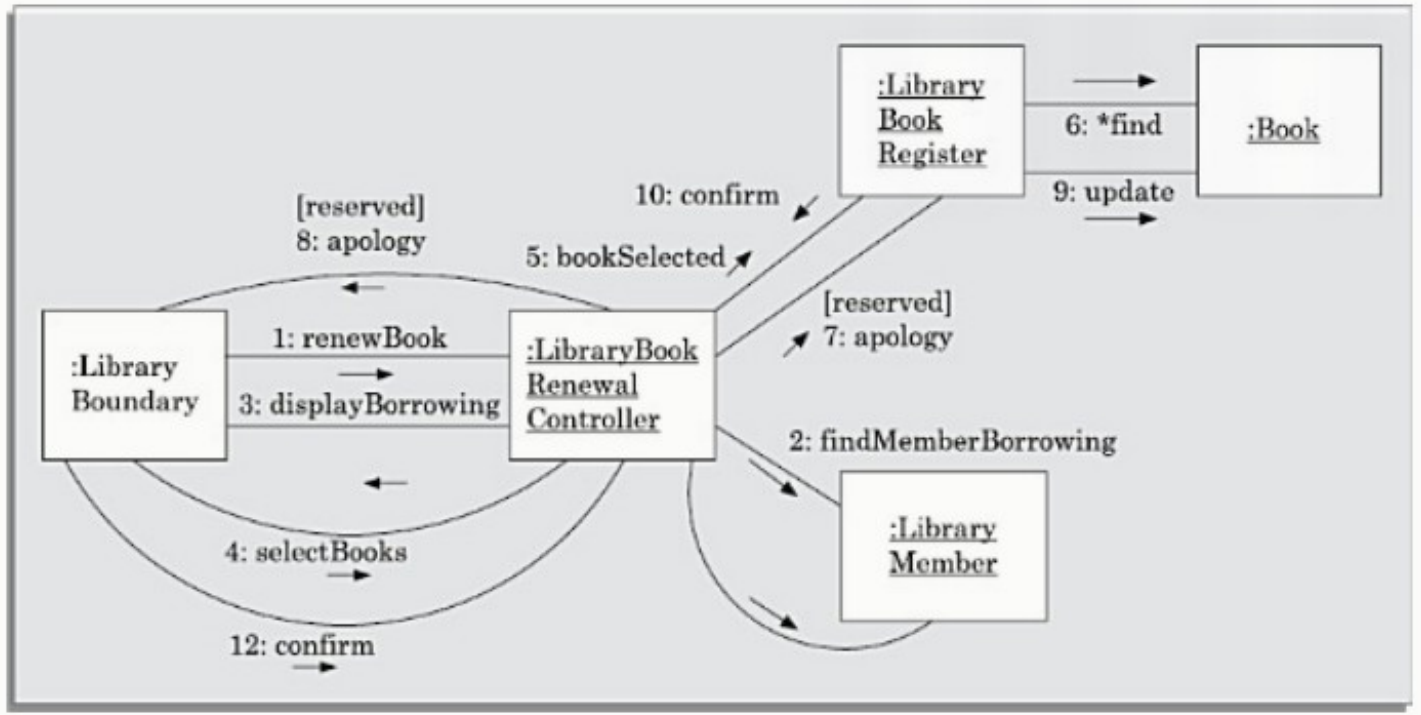


Figure 7.31: Collaboration diagram for the renew book use case.

- **The behavioural aspect is described by the set of messages** exchanged among the different collaborators.

- **The link between objects is shown as a solid line** and can be used to send messages between two objects.

- **The message is shown as a labelled arrow placed near the link**.

- **Messages are prefixed with sequence numbers.**

Activity Diagram

- The activity diagram **focuses on representing various activities or chunks of processing and their sequence of activation**.

- An **activity is a state with an internal action and one or more outgoing transitions** which automatically follow the termination of the internal activity.

- If an **activity has more than one outgoing transitions, then exact situation under which each is executed must be identified through appropriate conditions**

- **Activity diagrams support description of parallel activities and synchronisation aspects**

- **Parallel activities are represented on an activity diagram by using swim lanes. Swim lanes enable you to group activities based on who is performing them.**

- Activity diagrams can be **very useful to understand complex processing activities involving the roles played by many components**

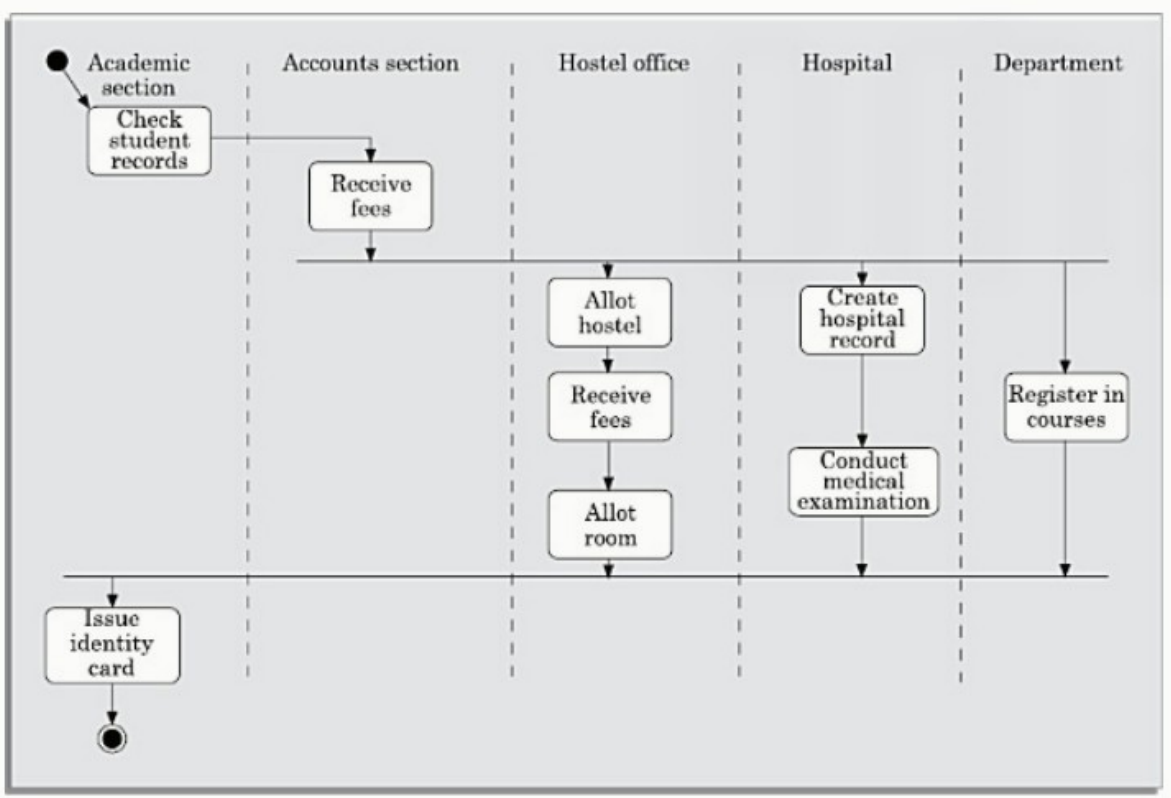


Figure 7.32: Activity diagram for student admission procedure at IIT.